RideMove

Creadores de Oportunidades

Laura Camila Diaz Rey & Fabian Sneider Martínez Moreno

Gestor del Conocimiento

Luis Ferney Ortiz Parra

Universidad de Cundinamarca

Ingeniería de Sistemas y Computación

Ingeniería de Software I

16 de septiembre de 2026

2026-2

Tabla de Contenido

1. Introducción del Proyecto 3
2. Descripción General del Sistema 5
2.1 Problema identificado 5
2.2 Objetivo del sistema 5
2.3 Usuarios del sistema 6
2.4 Alcance del proyecto 7
3. Enfoque Metodológico del Proyecto 8
3.1 Análisis del contexto del proyecto 8
3.2 Enfoque metodológico seleccionado 9
3.3 Uso de Scrum como marco de trabajo 10
4. Identificación y Clasificación de Requerimientos 11
4.1 Requerimientos Funcionales 11
4.2 Requerimientos No Funcionales 12
4.3 Priorización de Requerimientos 13
5. Dependencias del Proyecto 14
6. Configuración Tecnológica del Proyecto 15
7. Riesgos del Proyecto 17
8. Diseño de Base de Datos 18
8.1 Modelo Entidad–Relación (MER) 18
8.2 Modelo Relacional 19
8.3 Diccionario de Datos 19
9. Conclusiones del Proyecto 23

1. Introducción del Proyecto

En Fusagasugá, una parte importante del transporte urbano y semiurbano se coordina de manera informal a través de grupos de WhatsApp: una persona publica que necesita un servicio, espera a que algún conductor responda y, en muchos casos, debe además realizar una llamada para confirmar los detalles del viaje. Este proceso, aunque funcional, depende por completo de la disponibilidad y la atención de alguien que vea el mensaje a tiempo, lo que genera respuestas tardías, incertidumbre sobre el tiempo de espera y, en ocasiones, la necesidad de recurrir a otro medio de transporte.

RideMove nace como respuesta directa a esa situación. Es una aplicación orientada a la gestión y solicitud de servicios de transporte, pensada específicamente para el contexto de Fusagasugá y su modelo de transporte informal. El sistema no pretende sustituir a las plataformas comerciales de transporte que operan en grandes ciudades, sino organizar y agilizar un proceso que hoy se realiza de forma manual y dispersa, dándole estructura mediante una plataforma digital que centraliza las solicitudes, conecta a pasajeros con conductores disponibles y deja un registro confiable de cada servicio prestado.

El sistema se desarrollará como una aplicación de tres capas frontend, backend y base de datos, construida bajo el estilo de arquitectura hexagonal, en la que la lógica de negocio queda desacoplada de los detalles técnicos como el framework o el motor de base de datos. La comunicación entre el frontend y el backend se realizará mediante el protocolo HTTP, a través de una API, siguiendo el lineamiento definido para el proyecto integrador.

El objetivo general de este proyecto es desarrollar una primera versión funcional de RideMove que permita a un pasajero solicitar un servicio de transporte, a un conductor gestionar su disponibilidad y atender esas solicitudes, y a un administrador supervisar el funcionamiento general de la plataforma, aplicando en el proceso buenas prácticas de ingeniería de software: arquitectura hexagonal, principios SOLID, y una metodología de trabajo ágil apoyada en Scrum.

2. Descripción General del Sistema

2.1 Problema identificado

El problema identificado es la falta de un canal formal y confiable para solicitar servicios de transporte informal en Fusagasugá. Actualmente, este proceso depende de grupos de WhatsApp donde la solicitud se pierde entre otros mensajes, la respuesta del conductor no está garantizada ni acotada en el tiempo, y no existe ningún registro histórico de los servicios prestados, de las calificaciones de los conductores o de la información de contacto de los usuarios. Esto afecta tanto al pasajero, que no tiene certeza sobre si su solicitud será atendida, como al conductor, que no cuenta con una forma ordenada de ver y gestionar las solicitudes disponibles.

2.2 Objetivo del sistema

El objetivo del sistema es ofrecer una plataforma digital que organice el proceso de solicitud, aceptación, seguimiento y calificación de servicios de transporte, sustituyendo la coordinación manual por mensajes de texto por un flujo estructurado y trazable. De manera específica, el sistema busca:

•Permitir que un pasajero registre una solicitud de transporte indicando su lugar de recogida y destino.

•Permitir que un conductor consulte, acepte o rechace solicitudes disponibles según su disponibilidad.

•Mantener actualizada la ubicación del conductor para que pueda ser consultada por el sistema y por el pasajero durante el servicio.

•Dejar un registro histórico de los servicios prestados y de las calificaciones otorgadas.

•Brindar a un administrador visibilidad general sobre los usuarios, conductores y servicios registrados en la plataforma.

En términos generales, RideMove reemplaza el intercambio de mensajes y llamadas por un flujo estructurado y trazable dentro de la aplicación:

Solicitar → Buscar conductor disponible → Aceptar → Confirmar → Realizar servicio → Finalizar → Calificar

De esta manera, el pasajero tiene mayor claridad sobre lo que sucede con su solicitud, y el conductor cuenta con un espacio específico para consultar y gestionar los servicios que atiende.

2.3 Usuarios del sistema

El sistema contempla tres tipos de usuario, cada uno con necesidades y niveles de acceso distintos:

Tabla 1 Tipos de usuario del sistema

Tipo de usuario	Descripción
Pasajero	Persona que solicita un servicio de transporte, consulta el estado de su solicitud, visualiza información básica del conductor asignado y califica el servicio una vez finalizado.
Conductor	Persona que presta el servicio de transporte. Actualiza su disponibilidad y su ubicación, consulta las solicitudes disponibles, y las acepta, rechaza o actualiza hasta finalizar el servicio.
Administrador	Usuario encargado de gestionar la información general de la plataforma: usuarios, conductores y vehículos asociados, además de consultar reportes generales del funcionamiento del sistema.

2.4 Alcance del proyecto

Funcionalidades incluidas en esta primera versión

- Registro y gestión de perfil para pasajeros y conductores.
- Solicitud de servicios de transporte indicando origen y destino.
- Gestión de disponibilidad y de ubicación del conductor.
- Aceptación o rechazo de solicitudes por parte del conductor.
- Actualización y consulta del estado del servicio (pendiente, aceptado, en curso, finalizado).
- Calificación del servicio por parte del pasajero.
- Gestión administrativa básica de usuarios, conductores y vehículos.
- Almacenamiento persistente de toda la información anterior en una base de datos relacional.

Funcionalidades que no harán parte de esta primera versión

- Pagos electrónicos dentro de la aplicación.
- Integración con entidades bancarias.
- Cálculo automático y complejo de tarifas.
- Inteligencia artificial para la asignación automática de conductores.

3. Enfoque Metodológico del Proyecto

3.1 Análisis del contexto del proyecto

¿Los requerimientos pueden cambiar durante el proyecto?

Los requerimientos que describen el flujo principal —solicitud de servicio, disponibilidad del conductor, aceptación o rechazo, y actualización del estado del viaje— están claramente definidos y es poco probable que cambien en su esencia. Sin embargo, requerimientos secundarios como la gestión administrativa o los criterios exactos de calificación sí pueden ajustarse a medida que el equipo reciba retroalimentación del docente en cada comité, por lo que no se consideran completamente cerrados.

¿Existe incertidumbre técnica?

Sí. El equipo aplicará por primera vez arquitectura hexagonal de forma formal, lo que implica una curva de aprendizaje sobre cómo separar el dominio de negocio de los adaptadores técnicos. Tampoco se ha probado aún la integración completa entre el frontend, el backend y la base de datos, y existe incertidumbre puntual sobre la forma más adecuada de gestionar la ubicación de conductores y pasajeros dentro del sistema.

¿Qué riesgos iniciales identifican?

Se identifican riesgos técnicos (dificultad para gestionar correctamente la ubicación del conductor y su actualización periódica), riesgos de seguridad (manejo de contraseñas y datos personales), riesgos de proceso (complejidad en la lógica de asignación de conductores disponibles) y riesgos de equipo (tiempo y conocimiento técnico limitados, al ser un equipo de solo dos integrantes que además está aprendiendo Scrum y arquitectura hexagonal en paralelo).

¿El cliente podría modificar prioridades?

En este proyecto, el rol de cliente lo cumple el docente y el comité de arquitectura, quienes validan el avance en cada sesión y pueden solicitar ajustes de alcance o de prioridad entre una entrega y otra, tal como ya ocurrió al pasar de la aprobación inicial del proyecto a la exigencia de formalizar su metodología de trabajo. El equipo debe estar preparado para recibir estos ajustes de forma periódica.

3.2 Enfoque metodológico seleccionado

El equipo adopta un enfoque ágil basado en principios iterativo-incrementales. RideMove se construirá en ciclos cortos que entregan, cada uno, un incremento funcional del sistema —por ejemplo, primero el registro y los roles de usuario, luego la solicitud y gestión de servicios, y finalmente el historial, la calificación y la ubicación del conductor— en lugar de diseñar y entregar el sistema completo en una sola fase final.

Se eligió este enfoque porque un modelo predictivo exige requerimientos estables y baja incertidumbre técnica desde el inicio, condiciones que este proyecto no cumple del todo: existe incertidumbre técnica real en la arquitectura hexagonal y en el manejo de la ubicación, y el comité académico puede modificar prioridades entre entregas. Un enfoque ágil e iterativo-incremental permite aprender sobre la marcha, corregir el rumbo con cada entrega y mostrar avances concretos en cada comité.

Entre las ventajas de este enfoque para RideMove se destacan: la posibilidad de mostrar un incremento revisable en cada comité, la oportunidad de incorporar la arquitectura hexagonal de forma progresiva validando cada capa antes de construir la siguiente funcionalidad, y la facilidad para ajustarse a un equipo pequeño donde la comunicación es directa. Este enfoque ayuda a evitar

problemas como invertir tiempo en funcionalidades que luego deban cambiarse, o descubrir demasiado tarde que alguna parte de la integración técnica no funciona como se esperaba.

3.3 Uso de Scrum como marco de trabajo

Es importante diferenciar que Scrum no es la metodología de desarrollo del proyecto, sino el marco de trabajo mediante el cual el equipo organizará su día a día dentro del enfoque ágil iterativo-incremental ya seleccionado. Scrum ayudará a gestionar el proyecto a través de un backlog priorizado construido a partir de los requerimientos funcionales y no funcionales definidos y de sprints cortos con una duración estimada de una a dos semanas, cada uno con un objetivo claro y un incremento funcional entregable al final.

El backlog se organizará por prioridad: primero se abordan los requerimientos de prioridad alta, que cubren el flujo principal del servicio (solicitud, disponibilidad, aceptación y actualización de estado), y después los de prioridad media, como la gestión de perfil, la calificación y la gestión administrativa. Los eventos de Scrum que el equipo considera más importantes son:

- Sprint Planning: al inicio de cada sprint, para decidir qué elementos del backlog se desarrollarán y cómo se dividirá el trabajo entre Laura y Fabián.
- Daily Scrum: de forma breve e informal, dado que el equipo es de solo dos integrantes, para coordinar avances y destrabar dificultades técnicas.
- Sprint Review: alineado con los Comités de Arquitectura, donde se presenta el incremento funcional logrado y se recibe retroalimentación del docente.

- Retrospectiva: para identificar qué funcionó y qué ajustar en el proceso, especialmente relevante mientras el equipo aprende Scrum y arquitectura hexagonal de forma simultánea.

4. Identificación y Clasificación de Requerimientos

4.1 Requerimientos Funcionales

Tabla 2 Requerimientos funcionales del sistema

Código	Nombre del requerimiento	Descripción	Tipo de usuario asociado
RF-01	Gestión de perfil	El sistema debe permitir a los usuarios consultar y actualizar su información personal.	Pasajero / Conductor
RF-02	Solicitud de servicio	El sistema debe permitir al pasajero solicitar un servicio indicando el lugar de recogida y el destino.	Pasajero
RF-03	Disponibilidad del conductor	El sistema debe permitir al conductor indicar si se encuentra disponible para recibir solicitudes de servicio.	Conductor
RF-04	Consulta de solicitudes	El sistema debe permitir al conductor consultar las solicitudes de servicio que se encuentren disponibles.	Conductor
RF-05	Aceptación o rechazo del servicio	El sistema debe permitir al conductor aceptar o rechazar una solicitud de servicio disponible.	Conductor
RF-06	Actualización del estado del servicio	El sistema debe permitir al conductor actualizar el estado del servicio desde su inicio hasta su finalización.	Conductor
RF-07	Consulta del estado del servicio	El sistema debe permitir al pasajero consultar el estado de su solicitud y la información básica del conductor asignado.	Pasajero
RF-08	Calificación del servicio	El sistema debe permitir al pasajero calificar el servicio después de que este haya finalizado.	Pasajero
RF-09	Gestión administrativa	El sistema debe permitir al administrador gestionar usuarios y consultar información general sobre los servicios registrados.	Administrador

4.2 Requerimientos No Funcionales

Tabla 3 Requerimientos No funcionales del sistema

Categoría	Descripción	Criterio de validación
Seguridad	El sistema debe almacenar las contraseñas de los usuarios de forma cifrada y no como texto visible.	Al revisar la base de datos, las contraseñas no deben aparecer en texto plano.
Seguridad	El sistema debe controlar el acceso a las funciones según el rol del usuario.	Un pasajero no debe poder acceder a las funciones exclusivas del conductor o del administrador.
Rendimiento	El sistema debe responder a las operaciones principales en un tiempo máximo de 3 segundos bajo condiciones normales.	Pruebas de las operaciones principales verificando que la respuesta no supere los 3 segundos.
Escalabilidad	El sistema debe permitir agregar nuevos usuarios y servicios sin afectar las funciones principales existentes.	Pruebas agregando nuevos registros y verificación de que las funciones principales sigan operando correctamente.
Mantenibilidad	El backend debe mantener separada la lógica del negocio de los componentes externos mediante arquitectura hexagonal.	Revisión de que la lógica de dominio no dependa directamente de la base de datos ni del framework.
Usabilidad	La interfaz debe permitir que un pasajero realice una solicitud de servicio en no más de tres pasos.	Prueba de usabilidad cronometrando el número de pasos requeridos para completar una solicitud.

4.3 Priorización de Requerimientos

Tabla 4 Requerimientos funcionales priorizados

Código	Requerimiento	Prioridad
RF-01	Gestión de perfil	Media
RF-02	Solicitud de servicio	Alta
RF-03	Disponibilidad del conductor	Alta
RF-04	Consulta de solicitudes	Alta
RF-05	Aceptación o rechazo	Alta
RF-06	Actualización del estado	Alta
RF-07	Consulta del estado	Alta
RF-08	Calificación	Media
RF-09	Gestión administrativa	Media

Tabla 5 Requerimientos No funcionales priorizados

Requerimiento	Prioridad
Seguridad de contraseñas	Media
Seguridad por roles	Media
Rendimiento	Alta
Escalabilidad	Media
Mantenibilidad	Alta

El criterio utilizado para priorizar fue el valor que cada requerimiento aporta a la primera versión funcional del sistema. Se clasificaron como de prioridad alta los requerimientos que permiten completar el flujo principal del servicio de transporte desde que el pasajero solicita el viaje hasta que el conductor lo finaliza, incluyendo el manejo de la ubicación, ya que sin ella el pasajero no podría saber dónde se encuentra su conductor. Se clasificaron como de prioridad media aquellos requerimientos que aportan valor complementario, como el historial de calificaciones o la gestión administrativa, pero que no son indispensables para demostrar que la propuesta resuelve el problema inicial.

5. Dependencias del Proyecto

El desarrollo de RideMove depende de varios elementos externos e internos cuya disponibilidad condiciona el avance del proyecto. La siguiente tabla identifica las principales dependencias y su nivel de impacto sobre el desarrollo:

Tabla 6 Dependencias del proyecto

Dependencia	Descripción	Impacto
Base de datos (PostgreSQL)	Necesaria para almacenar usuarios, conductores, vehículos, solicitudes, ubicación y calificaciones. Sin ella no es posible persistir ninguna información del sistema.	Alto
Framework de backend (Spring Boot)	Sobre este framework se construyen los adaptadores de entrada y salida de la arquitectura hexagonal, así como la exposición de la API HTTP.	Alto
Conectividad a internet de los usuarios	Tanto el pasajero como el conductor requieren conexión activa para solicitar servicios, consultar el estado y actualizar la ubicación.	Alto

Dependencia	Descripción	Impacto
Framework de frontend (React)	Necesario para construir la interfaz con la que interactúan el pasajero y el conductor.	Medio
Librería de mapas (Leaflet / OpenStreetMap)	Utilizada para representar visualmente la ubicación del conductor y los puntos de recogida y destino.	Medio
Control de versiones (Git / GitHub)	Necesario para coordinar el trabajo entre los dos integrantes del equipo y mantener un historial ordenado de cambios.	Medio
Entorno de desarrollo (IntelliJ IDEA)	Herramienta utilizada por el equipo para el desarrollo del backend en Java.	Medio

6. Configuración Tecnológica del Proyecto

Como se definió en la visión de arquitectura inicial del proyecto, RideMove está organizado en tres componentes: el Frontend, con el que interactúan el pasajero y el conductor para mostrar el mapa, gestionar solicitudes y visualizar la información del servicio; el Backend, que concentra la lógica de negocio (validar usuarios, gestionar roles, crear y gestionar solicitudes, calificaciones y disponibilidad) y la expone mediante una API; y la Base de Datos, donde se almacena la información de usuarios, conductores, vehículos, ubicación, solicitudes y calificaciones. El equipo decidió organizar el Backend bajo Arquitectura Hexagonal, separando la lógica del negocio de los detalles técnicos, tal como se planteó como requisito del proyecto.

La siguiente tabla resume las tecnologías y decisiones técnicas definidas para el desarrollo. La arquitectura hexagonal y el protocolo de comunicación HTTP son lineamientos establecidos para el proyecto integrador y no son modificables por el equipo.

Tabla 7 Tecnologias

Componente	**Tecnología / Decisión**
Frontend	React 19.2
Backend	Java 21 + Spring Boot
Base de datos	PostgreSQL 18
Arquitectura de software	Arquitectura hexagonal (puertos y adaptadores)
Protocolo de comunicación	HTTP — API de tipo REST
Entorno de desarrollo	IntelliJ IDEA (Community Edition)
Control de versiones	Git y GitHub
Visualización de mapas y ubicación	Leaflet sobre datos de OpenStreetMap
Pruebas unitarias	JUnit 5 y Mockito

Como principio transversal de diseño, el backend se desarrollará aplicando los principios SOLID sobre la arquitectura hexagonal: cada caso de uso tendrá una única responsabilidad (SRP), el dominio dependerá de interfaces o puertos y no de implementaciones concretas (DIP), y los puertos se definirán de forma específica para cada necesidad en lugar de agrupar responsabilidades distintas en una sola interfaz (ISP). Esto permite que, por ejemplo, la forma en que se actualiza y se expone la ubicación del conductor pueda probarse de forma aislada, sin depender directamente del framework ni de la base de datos.

7. Riesgos del Proyecto

Tabla 8 Riesgos del proyecto

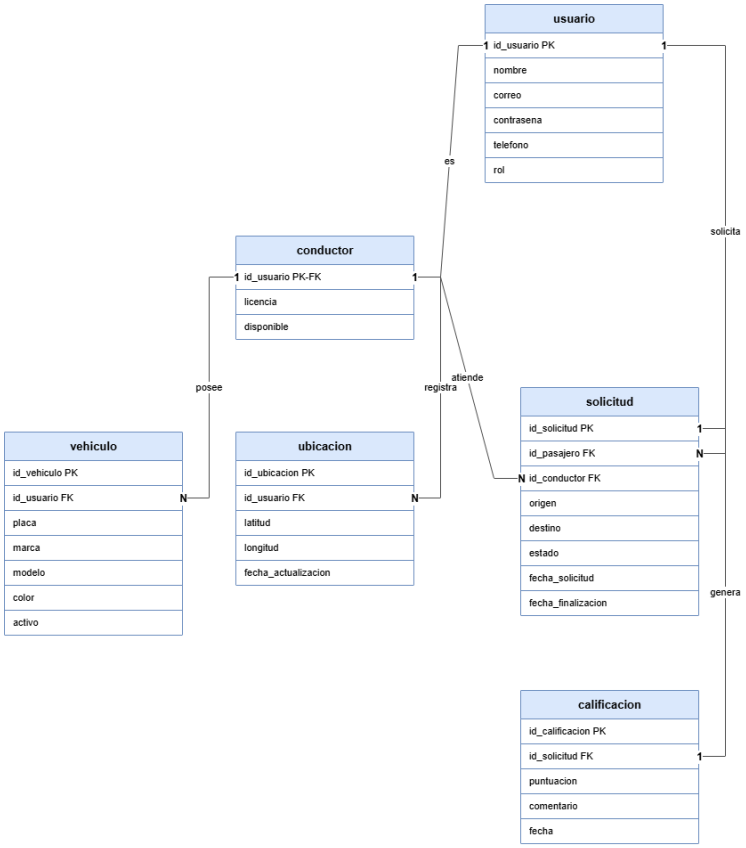
Riesgo	**Descripción**	**Nivel**
Manejo de la ubicación del conductor	Dificultad para implementar y gestionar correctamente la ubicación de conductores y pasajeros dentro de la arquitectura del sistema.	Medio
Seguridad de la información	Posible exposición de datos personales o contraseñas si no se implementan correctamente el cifrado y el control de acceso por rol.	Alto
Complejidad en la asignación de conductores	Dificultad para gestionar de forma ordenada la disponibilidad de varios conductores frente a múltiples solicitudes simultáneas.	Medio
Limitaciones de tiempo y experiencia técnica	El equipo, conformado por dos integrantes, está aprendiendo de forma simultánea Scrum y arquitectura hexagonal, lo que puede ralentizar el desarrollo.	Alto
Cambios de alcance o prioridades	El comité de arquitectura o el docente pueden solicitar ajustes de alcance entre una entrega y otra, como ya ha ocurrido previamente en el proyecto.	Medio

8. Diseño de Base de Datos

8.1 Modelo Entidad–Relación (MER)

El modelo entidad-relación de RideMove se construyó a partir de los requerimientos funcionales definidos en la sección 4. La entidad USUARIO concentra la información común de pasajeros, conductores y administradores; CONDUCTOR extiende esa información cuando el rol corresponde a un conductor, y de ella dependen sus vehículos y sus registros de ubicación. La entidad SOLICITUD conecta a un pasajero con, eventualmente, un conductor, y da lugar a una CALIFICACION una vez finalizado el servicio.

Figura 1 Modelo Entidad-Relación de la Base de Datos de RideMove



8.2 Modelo Relacional

A partir del modelo entidad-relación anterior, el modelo relacional de RideMove queda compuesto por las siguientes tablas:

Tabla 9 Modelo relacional de RideMove

Tabla	Descripción
usuario	Almacena la información común de todos los usuarios del sistema: pasajeros, conductores y administradores.
conductor	Extiende la información de un usuario cuando su rol es conductor, registrando su licencia y su disponibilidad.
vehiculo	Registra los vehículos asociados a un conductor.
ubicacion	Almacena las actualizaciones de ubicación reportadas por un conductor.
solicitud	Registra cada solicitud de servicio realizada por un pasajero, junto con el conductor que la atiende y su estado.
calificacion	Almacena la calificación y el comentario que un pasajero otorga a una solicitud ya finalizada.

8.3 Diccionario de Datos

Tabla 10 Diccionario de datos

Tabla	Campo	Tipo de dato	Descripción
usuario	id_usuario	INT (PK)	Identificador único del usuario.
usuario	nombre	VARCHAR(50)	Nombre completo del usuario.
usuario	correo	VARCHAR(60)	Correo electrónico usado para autenticación.

Tabla	Campo	Tipo de dato	Descripción
usuario	contraseña	VARCHAR(50)	Contraseña almacenada de forma cifrada.
usuario	telefono	VARCHAR(20)	Número de contacto del usuario.
usuario	rol	VARCHAR(20)	Rol del usuario: pasajero, conductor o administrador.
conductor	id_usuario	INT (PK, FK)	Referencia al usuario correspondiente.
conductor	licencia	VARCHAR(30)	Número de licencia de conducción.
conductor	disponible	BOOLEAN	Indica si el conductor está disponible para recibir solicitudes.
vehiculo	id_vehiculo	INT (PK)	Identificador único del vehículo.
vehiculo	id_usuario	INT (FK)	Conductor propietario del vehículo.
vehiculo	placa	VARCHAR(10)	Placa del vehículo.
vehiculo	marca	VARCHAR(30)	Marca del vehículo.
vehiculo	modelo	VARCHAR(10)	Modelo o año del vehículo.
vehiculo	color	VARCHAR(20)	Color del vehículo.
vehiculo	activo	BOOLEAN	Indica si es el vehículo activo del conductor.
ubicacion	id_ubicacion	INT (PK)	Identificador único del registro de ubicación.
ubicacion	id_usuario	INT (FK)	Conductor al que pertenece la ubicación.

Tabla	Campo	Tipo de dato	Descripción
ubicacion	latitud	DECIMAL(9,6)	Coordenada de latitud reportada.
ubicacion	longitud	DECIMAL(9,6)	Coordenada de longitud reportada.
ubicacion	fecha_actualizacion	TIMESTAMP	Momento en que se registró la ubicación.
solicitud	id_solicitud	INT (PK)	Identificador único de la solicitud.
solicitud	id_pasajero	INT (FK)	Usuario pasajero que realiza la solicitud.
solicitud	id_conductor	INT (FK)	Conductor asignado a la solicitud (si aplica).
solicitud	origen	VARCHAR(150)	Lugar de recogida indicado por el pasajero.
solicitud	destino	VARCHAR(150)	Destino indicado por el pasajero.
solicitud	estado	VARCHAR(20)	Estado del servicio: pendiente, aceptado, en curso o finalizado.
solicitud	fecha_solicitud	TIMESTAMP	Fecha y hora en que se creó la solicitud.
solicitud	fecha_finalizacion	TIMESTAMP	Fecha y hora en que finalizó el servicio.
calificacion	id_calificacion	INT (PK)	Identificador único de la calificación.
calificacion	id_solicitud	INT (FK)	Solicitud a la que pertenece la calificación.
calificacion	puntuacion	INT	Valor numérico otorgado por el pasajero (1 a 5).

Tabla	Campo	Tipo de dato	Descripción
calificacion	comentario	VARCHAR(255)	Comentario opcional del pasajero.
calificacion	fecha	TIMESTAMP	Fecha en que se registró la calificación.

9. Conclusiones del Proyecto

El desarrollo de este documento consolidado permitió al equipo pasar de una idea general organizar el transporte informal en Fusagasugá a una propuesta técnica concreta, con requerimientos definidos, una arquitectura justificada, un enfoque metodológico argumentado y un modelo de datos completo. Este proceso deja claro que, en la industria del software, ninguna decisión técnica se toma de forma aislada: el enfoque metodológico condiciona cómo se organiza el trabajo, la arquitectura condiciona cómo evoluciona el sistema, y el modelo de datos condiciona qué tan bien se sostiene todo lo anterior en el tiempo.

Entre los principales desafíos encontrados durante este proceso se destacan aprender a aplicar arquitectura hexagonal y principios SOLID de forma simultánea con Scrum, sin haber trabajado antes con ninguno de los dos de manera formal; definir con precisión cómo se representará y gestionará la ubicación de conductores y pasajeros dentro del sistema; y mantener el ritmo de trabajo esperado siendo un equipo de solo dos integrantes con tiempo académico limitado.

En cuanto a la aplicación de conceptos de ingeniería de software, el equipo utilizó un análisis de contexto formal para justificar la elección de un enfoque ágil iterativo-incremental, construyó un backlog priorizado a partir de requerimientos funcionales y no funcionales clasificados con criterios explícitos, y diseñó un modelo entidad-relación y un diccionario de datos completos antes de escribir la primera línea de código del backend. Esta forma de trabajar, más ordenada que la de un desarrollo improvisado, es precisamente lo que se espera de un equipo que aplica ingeniería de software de manera profesional, y sienta una base sólida para las siguientes etapas de construcción de RideMove.